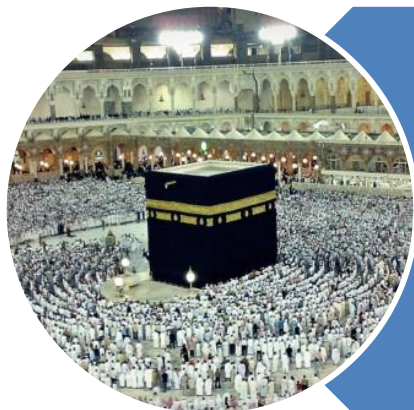


رَبِّ اشْرَحْ لِي صَدْرِي وَيَسِّرْ لِي أَمْرِي وَاحْلُلْ عُقْدَةً مِنْ لِسَانِي يَفْقَهُوا قَوْلِي



O my Lord! Expand for me my chest [with assurance] and ease for me my task and untie the knot from my tongue that they may understand my speech. **(Quran 20 : 25-28)**

Recursion

with “String” values

Printing String Character by Character

Example 01

C

P

C

S

2

0

4

4

Printing String - Introduction

- String is an **array of characters**
- Character reference start from **“0”** till **“n-1”**
- Accessing **first** character of string **“str”**
 - **str.charAt(0)**
- Extracting the substring starting from 2nd character till the last one
 - **str.substring(1)**
- Extracting the substring from **“ith”** to **“jth”** character
 - **str.substring(i, j+1)**

C

P

C

S

2

0

4

5

Printing String— **Generic Formula**

- Printing character
 - **str . charAt (0)**
- Updating string for next recursive call
 - **str . Substring(1)**
- Printing method will be represented by
 - **printChar(str)**

C

P

C

S

2

0

4

6

Printing String – Base Case

- The printing should continue until the string become null
 - Str == **null** then stop

C

P

C

S

2

0

4

7

Printing String – Code

- Initial Method Call

printChar(str)

- Base Case

str == null

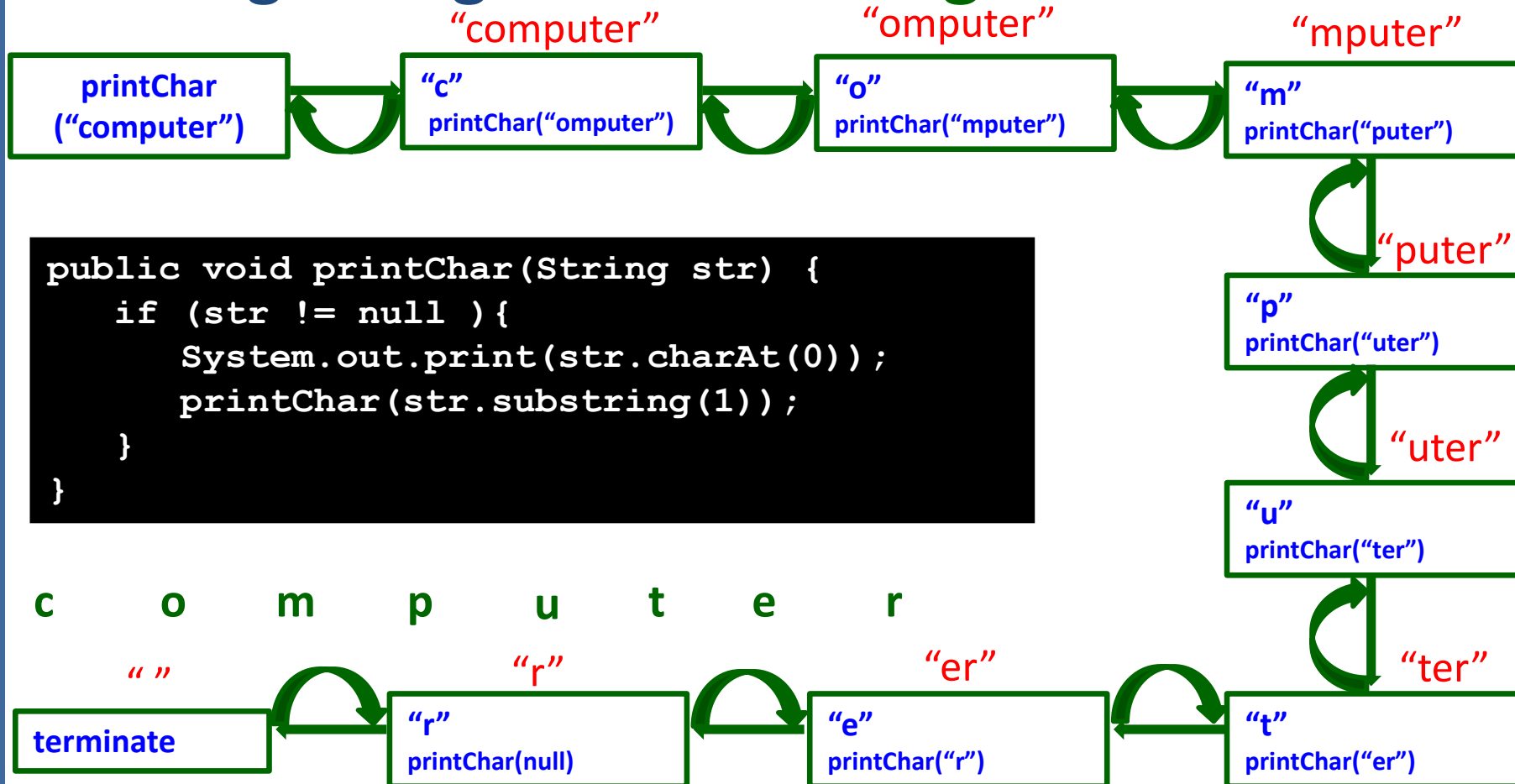
```
public void printChar(String str) {  
    if (str != null ) {  
        System.out.print(str.charAt(0));  
        printChar(str.substring(1));  
    }  
}
```

Generic Formula

printChar(str.substring(1))

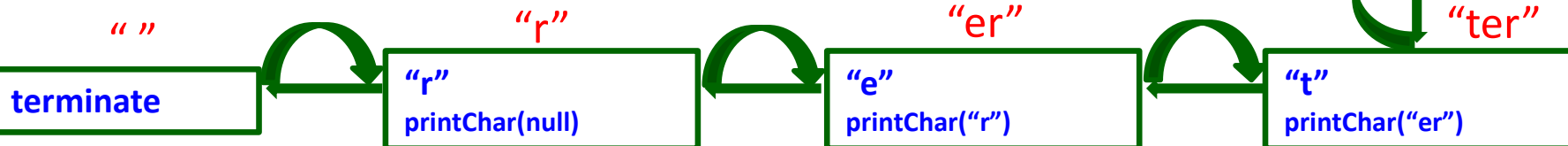
Printing String – Code Tracing

printChar ("computer")



```
public void printChar(String str) {
    if (str != null ) {
        System.out.print(str.charAt(0)) ;
        printChar(str.substring(1)) ;
    }
}
```

c o m p u t e r



Printing String in Reverse

Example 02

Printing String in Reverse - **Introduction**

- Input String
 - e.g. “computer”
- Output of program
 - r e t u p m o c

C

P

C

S

2

0

4

Printing String in Reverse – Code

- Initial Method Call

printChar(str)

- Base Case

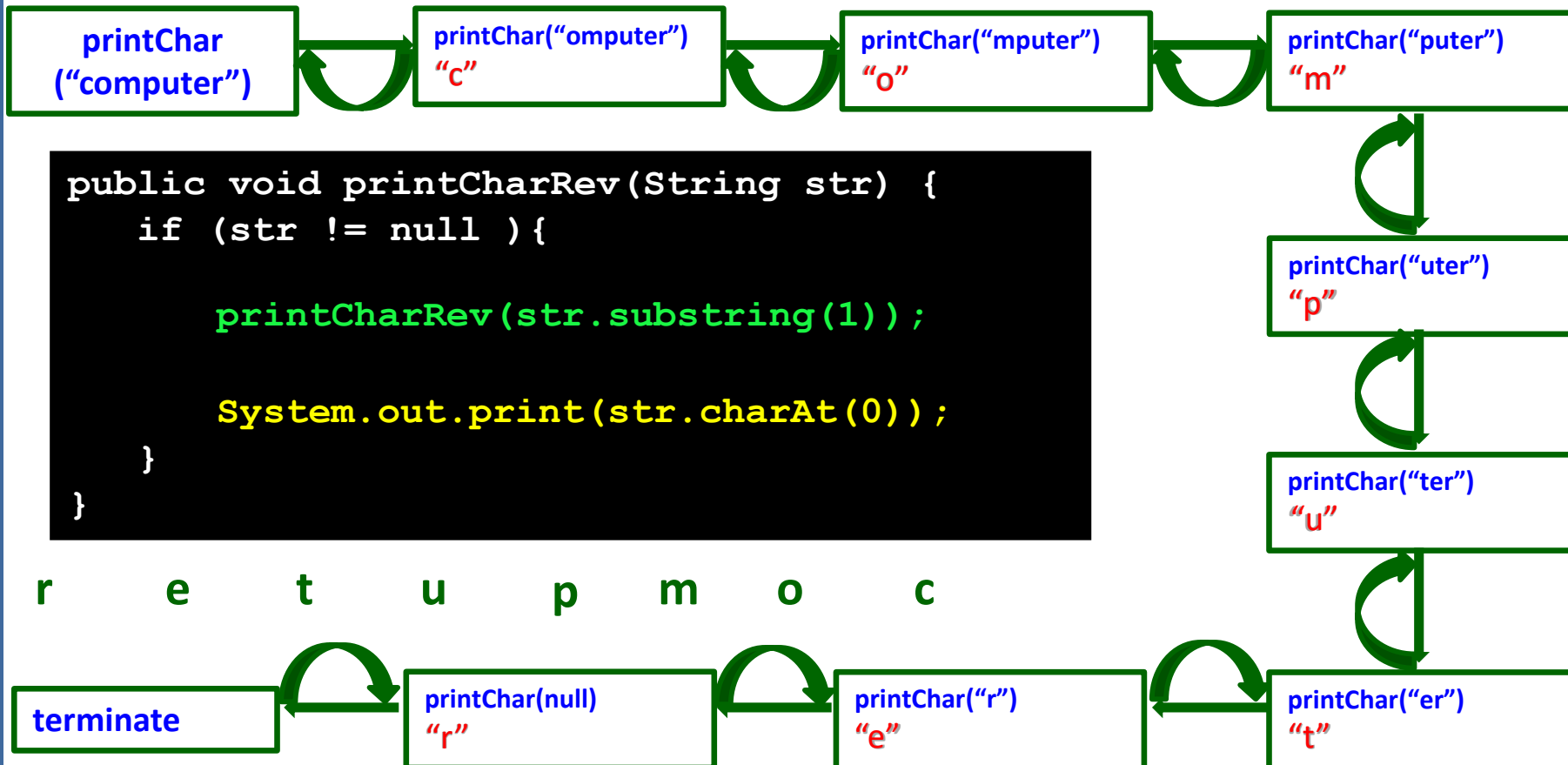
str = null

```
public void printCharRev(String str) {  
    if (str != null ) {  
        printCharRev(str.substring(1));  
        System.out.print(str.charAt(0));  
    }  
}
```

Generic Formula

printCharRev(str.substring(1))

Printing String in Reverse — Code Tracing `printCharRev("computer")`



Counting the occurrence of character “x” in a given string

Example 03

C

P

C

S

2

0

4

Counting “x” – Code

- Initial Method Call

countX(str)

- Base Case

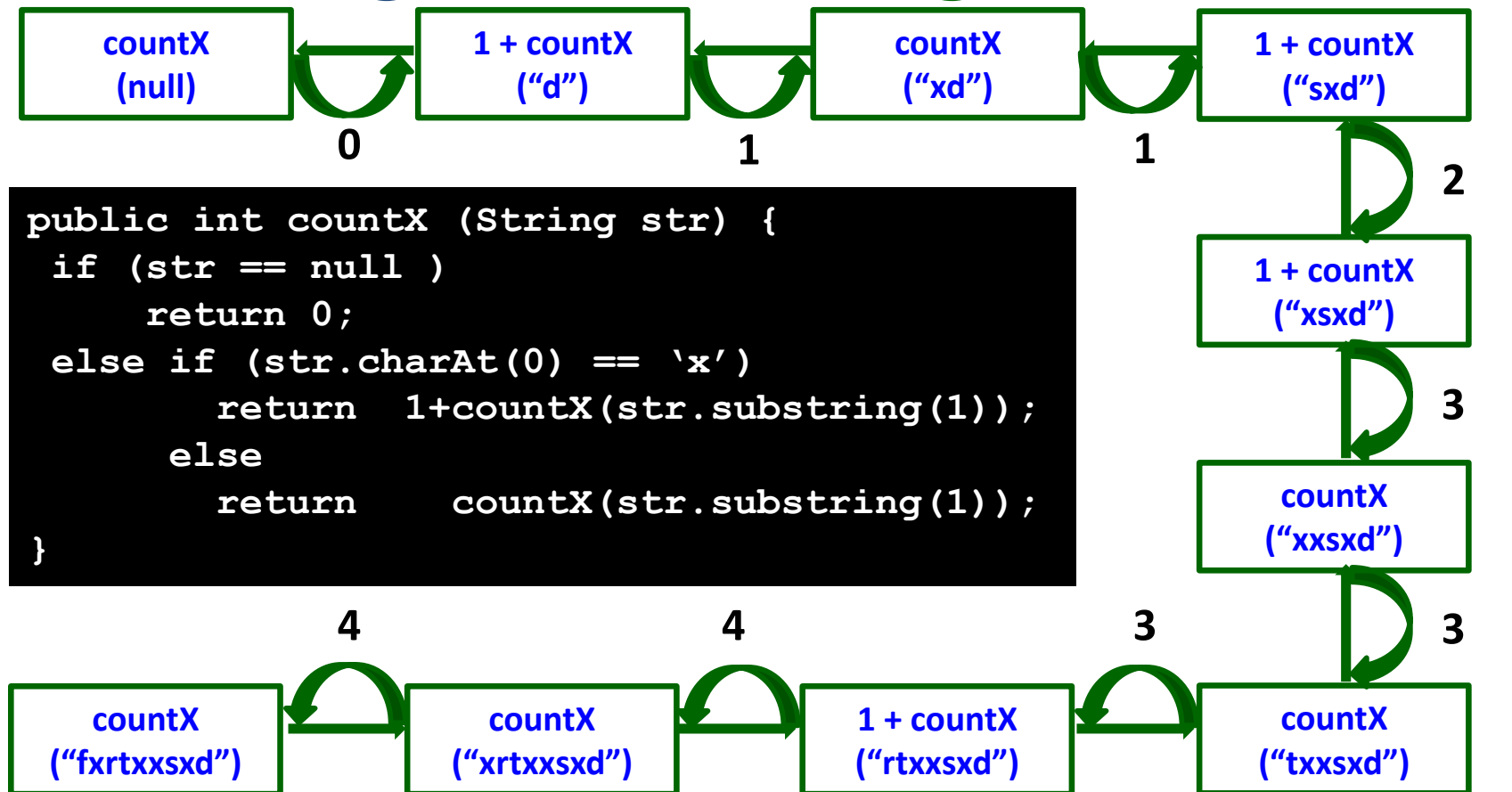
str = null

```
public int countX (String str) {
    if (str == null )
        return 0;
    else if (str.charAt(0) == 'x')
        return 1+countX(str.substring(1)) ;
    else
        return countX(str.substring(1)) ;
}
```

Generic Formula

countX(str.substring(1))

Counting “x” – Tracing



Change all occurrences of “x”
with “y” in a given string

Example 04

C

P

C

S

2

0

4

Change "x" with "y" – Code

- Initial Method Call

changeXY(str)

- Base Case

str = null

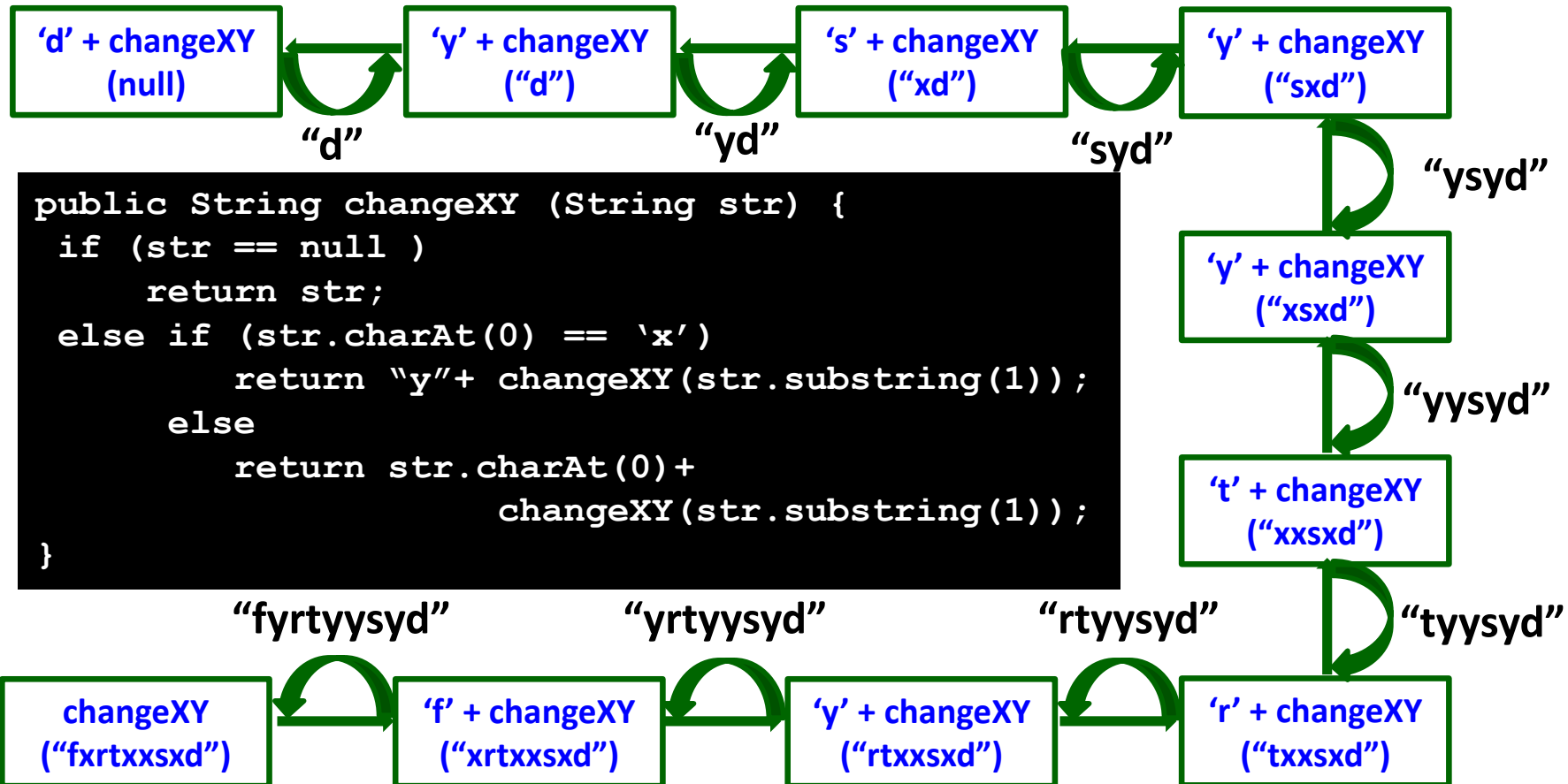
```
public String changeXY (String str) {
    if (str == null )
        return str;
    else if (str.charAt(0) == 'x')
        return "y"+ changeXY(str.substring(1)) ;
    else
        return str.charAt(0)+
            changeXY(str.substring(1)) ;
}
```

Generic Formula

changeXY (str.substring(1))

Change “x” with “y” – Tracing

changeXY(“fxrtxxsxd”)



Result = “fyrtyysyd”

**Remove all occurrences of “x” in
a given string**

Example 05

C

P

C

S

2

0

4

Remove “x” – Code

- Initial Method Call

removeX(str)

- Base Case

str = null

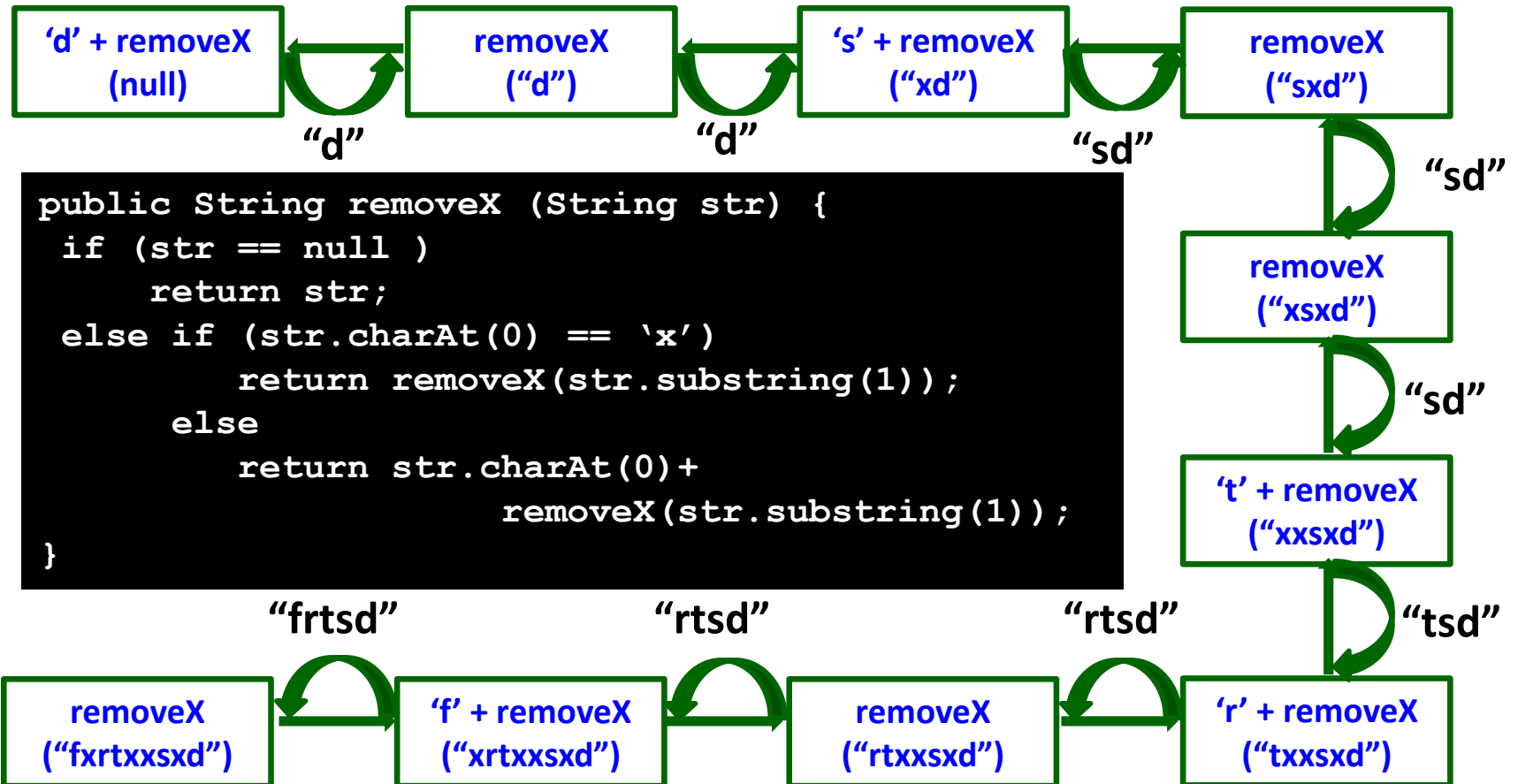
```
public String removeX (String str) {
    if (str == null )
        return str;
    else if (str.charAt(0) == 'x')
        return removeX(str.substring(1));
    else
        return str.charAt(0) +
            removeX(str.substring(1));
}
```

Generic Formula

removeX(str.substring(1))

Remove “x” – Tracing

removeX(“fxrtxxsxd”)



Result = “frtsd”

Recursion

Major Applications

Binary Search

Example 01

Binary Search - Iterative

index	0	1	2	3	4	5	6	7	8
value	2	6	19	27	33	37	38	41	118

- Searching the value, **19**
- Find the middle **INDEX**
 - The lowest index is 0
 - The highest index is 8
 - So the middle **(low+high)/2** index is 4
- **Three** possibilities
 - Value is **equal** Search **Successful**
 - Value is **smaller** Search in **lower half**; **high = mid - 1**
 - Value is **larger** Search in **upper half**; **low = mid + 1**

Binary Search – Iterative(Code)

```
public static int binSearch(int[] a, int value) {  
    int low = 0;  
    int high = a.length;  
    while (low <= high) {  
        int mid = (low + high)/2;  
        if (a[mid] == value)  
            return 1; // return 1 if found  
        else if (a[mid] < value)  
            low = mid + 1;  
        else  
            high = mid - 1;  
    }  
    return 0; // this only happens if not found  
}
```

C

P

C

S

2

0

4

Binary Search – Recursive

- Base Case
 - **low > high**
- $mid = (low + high) / 2$
- If $value < a[mid]$
 - `binSearch(a, low, mid-1, val)`
- If $value > a[mid]$
 - `binSearch(a, mid+1, high, val)`
- If $a[mid] = value$
 - stop **searching**

C

Binary Search – Recursive(Code)

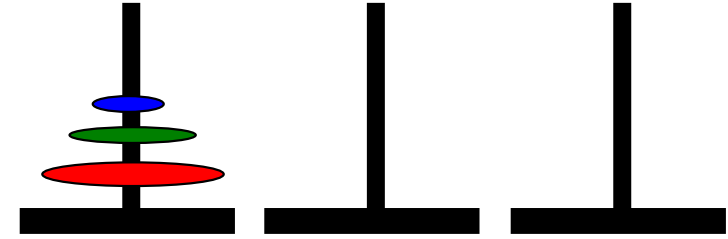
```
public static int binSearch(int[] a, int low, int high, int Val){  
    int mid;  
    if (low <= high) {  
        mid = (low+high)/2;  
        if (Val < a[mid])  
            return binSearch(a, low, mid-1, Val);  
        else if (Val > a[mid])  
            return binSearch(a, mid+1, high, Val);  
        else  
            return 1;  
    }  
    return 0;  
}
```

4

Tower of Hanoi

Example 02

Towers of Hanoi



- There are three vertical poles
- There are 64 disks on tower 1 (left most tower)
 - The disks are arranged with the largest diameter disks at the bottom
- Some monk has the daunting task of moving disks from one tower to another tower
 - Often defined as moving from Tower #1 to Tower #3
 - Tower #2 is just an intermediate pole
 - He can only move ONE disk at a time
 - And he MUST follow the rule of NEVER putting a bigger disk on top of a smaller disk

C

P

C

S

2

0

4

Towers of Hanoi - **Solution**

- Irrespective of the number of disks, the following steps **MUST** be carried out:
 - The bottom disk needs to move to the destination tower
 - 1) So step 1 must be to move all disks above the bottom disk to the intermediate tower
 - 2) In step 2, the bottom disk can now be moved to the destination tower
 - 3) In step 3, the disks that were initially above the bottom disk must now be put back on top
 - Of course, at the destination
- Let's look at the situation with only 3 disks...

C

P

C

S

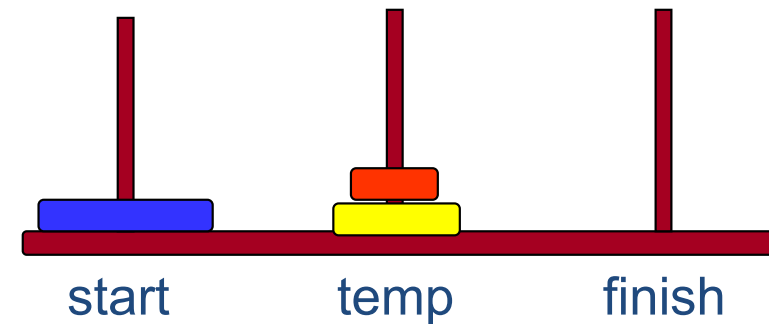
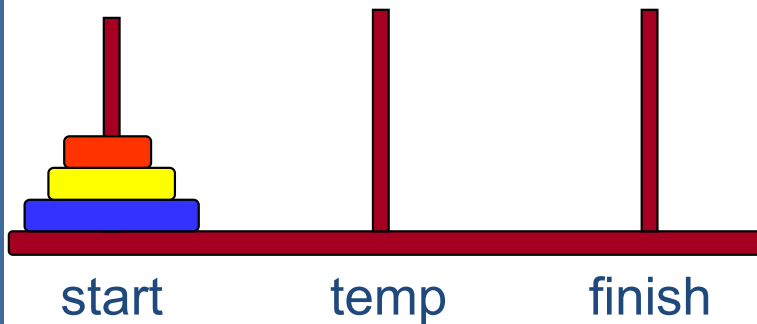
2

0

4

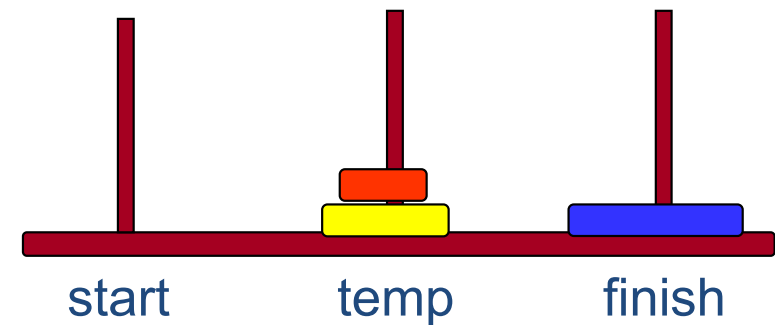
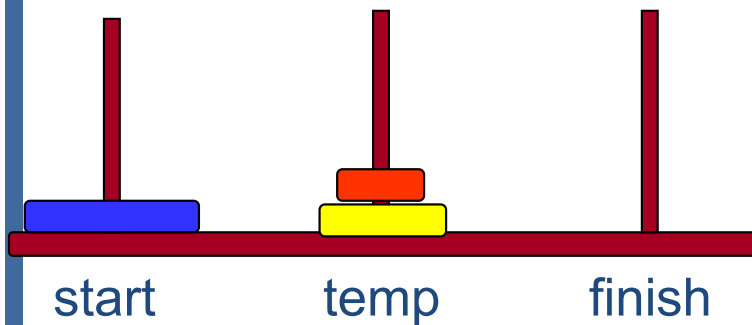
Towers of Hanoi – **Solution** step-1

- Move 2 disks from start to temp using finish Tower.
- To understand the recursive routine, let us assume that we know how to solve 2 disk problem, and go for the next step.
 - Meaning, we “know” how to move 2 disks appropriately



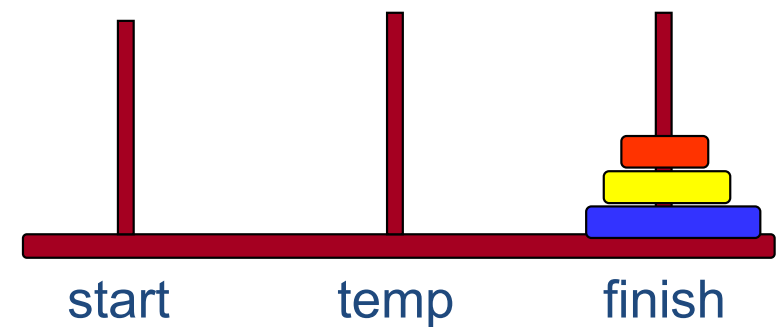
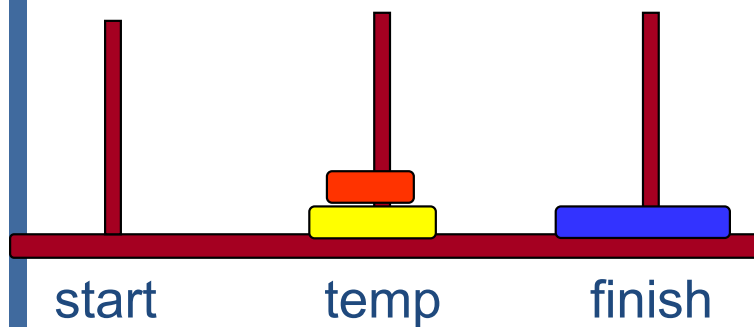
Towers of Hanoi – **Solution** step-2

- Move the (remaining) single disk from start to finish
- This does not involve recursion
 - and can be carried out without using temp tower.
- In our program, this is just a print statement
 - Showing what we moved and to where



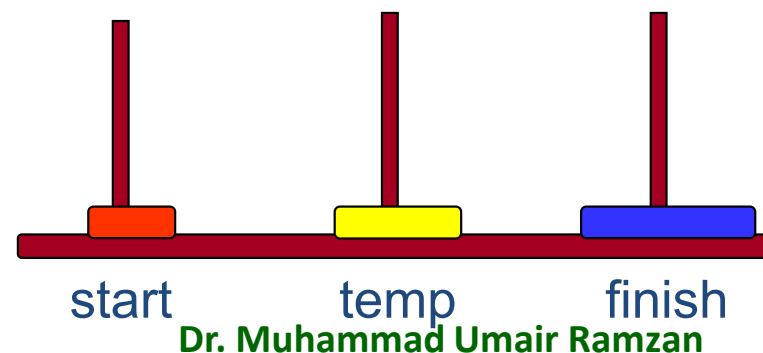
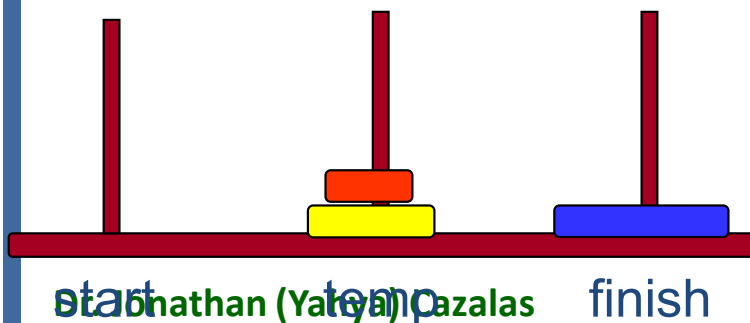
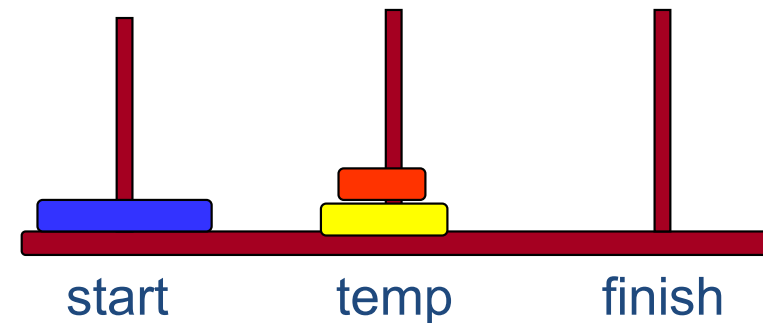
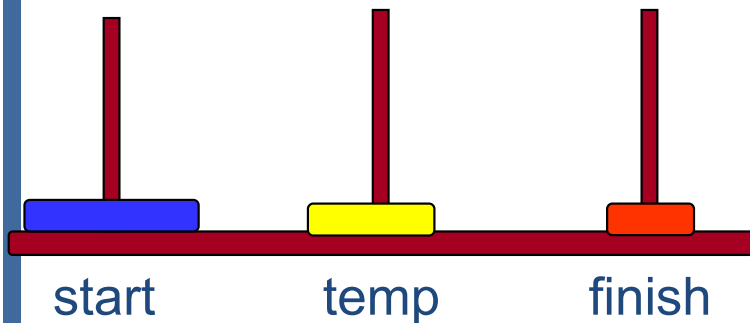
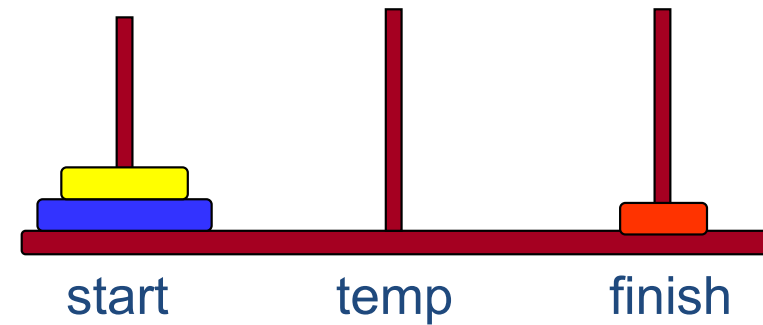
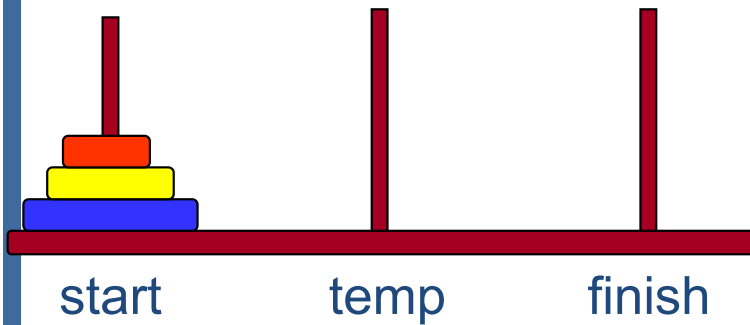
Towers of Hanoi – **Solution** step-3

- Now we are at the last step of the routine.
- Move the 2 disks from temp tower to finish tower using the start tower
 - This is also done recursively

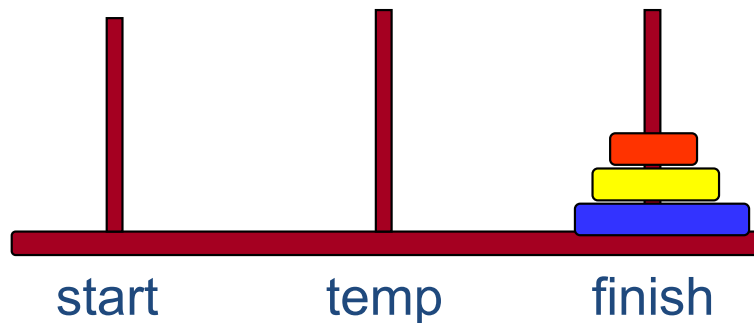
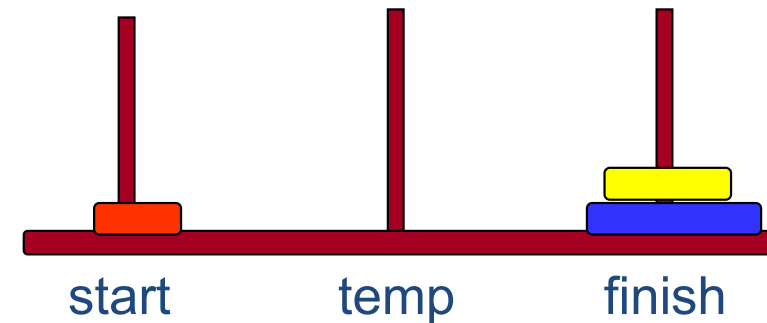
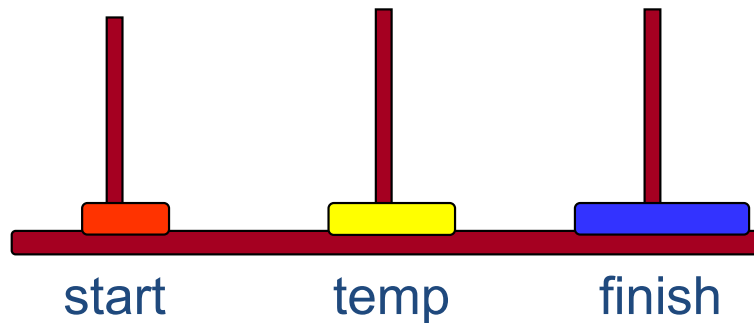


Towers of Hanoi – Complete Solution

C
P
C
S
2
0
4



Towers of Hanoi – Complete Solution



■ # of steps needed:

- We had 3 disks requiring 7 steps
- 4 disks would require 15 steps
- n disks would require $2^n - 1$ steps
- HUGE number

Towers of Hanoi – Iterative

```
// C Program for Iterative Tower of Hanoi
#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <limits.h>

// A structure to represent a stack
struct Stack
{
    unsigned capacity;
    int top;
    int *array;
};

// function to create a stack of given capacity.
struct Stack* createStack(unsigned capacity)
{
    struct Stack* stack =
        (struct Stack*) malloc(sizeof(struct Stack));
    stack->capacity = capacity;
    stack->top = -1;
    stack->array =
        (int*) malloc(stack->capacity * sizeof(int));
    return stack;
}

// Stack is full when top is equal to the last index
int isFull(struct Stack* stack)
{
    return (stack->top == stack->capacity - 1);
}

// Stack is empty when top is equal to -1
int isEmpty(struct Stack* stack)
{
    return (stack->top == -1);
}
```

```
return (stack->top == -1);
}

// Function to add an item to stack. It increases
// top by 1
void push(struct Stack *stack, int item)
{
    if (isFull(stack))
        return;
    stack->array[++stack->top] = item;
}

// Function to remove an item from stack. It
// decreases top by 1
int pop(struct Stack* stack)
{
    if (isEmpty(stack))
        return INT_MIN;
    return stack->array[stack->top--];
}

// Function to implement legal movement between
// two poles
void moveDisksBetweenTwoPoles(struct Stack *src,
    struct Stack *dest, char s, char d)
{
    int pole1TopDisk = pop(src);
    int pole2TopDisk = pop(dest);

    // When pole 1 is empty
    if (pole1TopDisk == INT_MIN)
    {
        push(src, pole2TopDisk);
        moveDisk(d, s, pole2TopDisk);
    }
}
```

```
// When pole2 pole is empty
else if (pole2TopDisk == INT_MIN)
{
    push(dest, pole1TopDisk);
    moveDisk(s, d, pole1TopDisk);
}

// When top disk of pole1 > top disk of pole2
else if (pole1TopDisk > pole2TopDisk)
{
    push(src, pole1TopDisk);
    push(src, pole2TopDisk);
    moveDisk(d, s, pole2TopDisk);
}

// When top disk of pole1 < top disk of pole2
else
{
    push(dest, pole2TopDisk);
    push(dest, pole1TopDisk);
    moveDisk(s, d, pole1TopDisk);
}

//Function to show the movement of disks
void moveDisk(char fromPeg, char toPeg, int disk)
{
    printf("Move the disk %d from '%c' to '%c'\n",
        disk, fromPeg, toPeg);
}

//Function to implement TOH puzzle
void tohIterative(int num_of_disks, struct Stack
    *src, struct Stack *aux,
```

Towers of Hanoi – Iterative

```

        struct Stack *dest)
    {
        int i, total_num_of_moves;
        char s = 'S', d = 'D', a = 'A';

        //If number of disks is even, then interchange
        //destination pole and auxiliary pole
        if (num_of_disks % 2 == 0)
        {
            char temp = d;
            d = a;
            a = temp;
        }
        total_num_of_moves = pow(2, num_of_disks) - 1;

        //Larger disks will be pushed first
        for (i = num_of_disks; i >= 1; i--)
            push(src, i);

        for (i = 1; i <= total_num_of_moves; i++)
        {
            if (i % 3 == 1)
                moveDisksBetweenTwoPoles(src, dest, s, d);

            else if (i % 3 == 2)
                moveDisksBetweenTwoPoles(src, aux, s, a);

            else if (i % 3 == 0)
                moveDisksBetweenTwoPoles(aux, dest, a, d);
        }
    }

```

```

// Driver Program
int main()
{
    // Input: number of disks
    unsigned num_of_disks = 3;

    struct Stack *src, *dest, *aux;

    // Create three stacks of size 'num_of_disks'
    // to hold the disks
    src = createStack(num_of_disks);
    aux = createStack(num_of_disks);
    dest = createStack(num_of_disks);

    tohIterative(num_of_disks, src, aux, dest);
    return 0;
}

```

C

P

C

S

2

0

4

Towers of Hanoi – Recursive

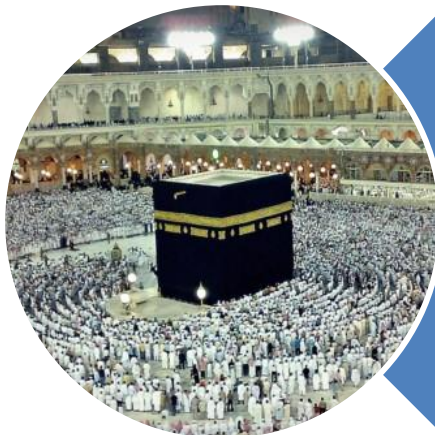
- Tower(N, BEG, AUX, END)
 - If $N = 1$ Then
 - Move Disk from “Beg” to “End”
 - Return
 - Tower (N-1, Beg, End, Aux)
 - Move Disk from “Beg” to “End”
 - Tower (N-1, Aux, Beg, End)
 - Return

Recursion

Some Examples

<https://codingbat.com/java/Recursion-1>

<https://codingbat.com/java/Recursion-2>



Allah (Alone) is Sufficient for us,
and He is the Best Disposer of
affairs (for us). **(Quran 3 : 173)**